Accelerating Monte Carlo Pricing of Asian Options using Neural Networks

Yashivi Bhatt Yaru Guan Yue Wu Zhengtao Yu

University of Illinois at Urbana–Champaign
Instructor: Matthew Murphy

December 2025

Outline

1. Introduction
2. Methodology
3. Experimental Results
4. Impact of Data Simulation on Model Performance
5. Takeaway

Problem Motivation

General Motivation

- Asian options are **path-dependent**, relying on the full price trajectory.
- Fast, scalable pricing is essential for trading and risk systems.
- **Goal:** develop a fast and accurate surrogate for Monte Carlo pricing.

Limitations of Classical Methods

- **PDE**: requires an extra state variable $\rightarrow$ high-dimensional and costly.
- **Monte Carlo**: accurate but **slow**; requires many paths and yields noisy Greeks.

Why Neural Networks?

- Learn **sequence-to-scalar** mappings for path-dependent payoffs.
- Provide fast inference and smooth Greeks.
- Aim: examine architectures and how **data generation** affects performance.

Basic Settings

Pricing Framework

- Underlying follows risk–neutral **GBM**.
- We price **arithmetic Asian call options**.

Parameter Space

- Uniform sampling over:

$$S_0, K \in [80, 120],\ \sigma \in [0.10, 0.40],\ r \in [0, 0.05],\ q \in [0, 0.02],\ T = 1.$$

- S_0 and K sampled independently to span full moneyness.

Data Construction

- Simulate **50-step GBM paths**; Monte Carlo provides ground-truth prices.
- NN input: normalized path (S_t/K) + moneyness K/S_0.

EDA Summary Statistics

Payoff Distribution

- 30,000 simulated samples; 52.9% zero payoffs, 47.1% positive payoffs.
- Mean = 0.1975, Median = 0.1541, Std = 0.1740; Range: [0, 1.8362].

Moneyness Coverage

- ITM 41.7%, OTM 37.3%, ATM 21.0%.
- Ensures broad and balanced representation across pricing regimes.

Independence Validation

- Parameter correlations are negligible: $|\mathrm{Corr}(\sigma, \cdot)| < 0.174$, $|\mathrm{Corr}(r, \cdot)| < 0.070$, $|\mathrm{Corr}(q, \cdot)| < 0.038$.
- $|\mathrm{Corr}(S_0, K)| = 0.049$ confirms independent sampling.

Takeaway

- Dataset is balanced, diverse, and statistically independent, suitable for robust model training.

Methodology: Training Setup

Learning Framework

- Input: normalized 50-step path (S_t/K) + moneyness (K/S_0).
- Target: normalized price V/K (and Delta for joint learning).
- Ground truth from high-precision Monte Carlo.

Training

- Loss: MSE; Optimizer: Adam.
- Split: 70% train / 15% val / 15% test.
- Early stopping to ensure stability.

Evaluation

- Metrics: MAE, RMSE, percentile error, tail error.
- Delta accuracy for hedging relevance.

Methodology: MLP and CNN

MLP (Baseline)

- Fully connected network; no temporal modeling.

CNN

- 1D convolutions capture **local temporal patterns**.
- Effective for Asian options where averaging depends on short-term movements.
- Provides stable feature extraction across moneyness.

Role

- Tests whether local structure alone can approximate MC pricing.

Methodology: LSTM and CNN–LSTM Hybrid

LSTM

- Models **long-range dependencies** across the full path.
- Suitable for Asian payoffs that aggregate all time steps.

CNN–LSTM Hybrid

- CNN extracts local patterns; LSTM adds global memory.
- Tests whether combining both structures improves pricing accuracy.

Purpose

- Evaluate the need for both local and long-range temporal modeling.

Methodology: TCN and Comparison Rationale

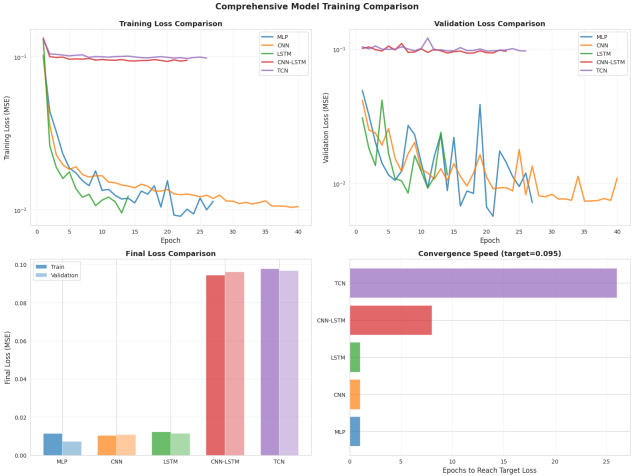
TCN

- Dilated convolutions expand temporal receptive field.
- Parallelizable alternative to recurrent models.

Comparison Rationale

- Models represent different ways to learn path-dependent structure: local (CNN), global (LSTM), hybrid, dilated (TCN), and baseline (MLP).
- Enables identifying which temporal mechanisms best approximate MC pricing.

Model Training Comparison: Finding the Winner



Setup:

- 5 architectures tested
- 30,000 training samples
- 50 epochs max

Key Findings:

- **CNN & LSTM** converge fastest (2–4 epochs)
- **CNN-LSTM & TCN** struggle (10–26 epochs)
- Final errors: **CNN 1.1%, LSTM 1.2%**

Adding Greeks: Joint Price & Delta Prediction

Why Delta Matters:

- Real-time hedging requires Greeks
- MC computes delta slowly
- Can NN predict *both* price & delta?
- Enables real-time market making & portfolio risk updates

Experimental Design:

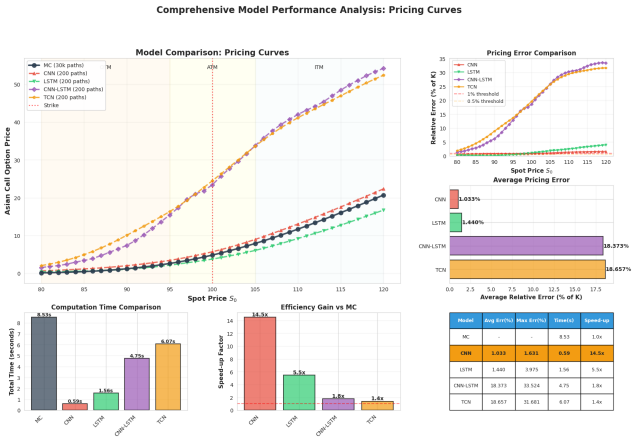
- Multi-task learning framework
- 4 test scenarios (ATM/ITM/OTM)
- Compare CNN vs LSTM

Results: CNN vs LSTM

Model	Scenario	Price Err	Δ Err	Speed-up
CNN	ATM	0.90%	4.00%	20×
CNN	ITM, high vol	0.52%	6.27%	19.6×
LSTM	ATM	1.04%	20.38%	6.1×
LSTM	OTM high vol	0.00%	20.78%	6.1×

CNN: 4–6% delta error with 20× speedup
LSTM: >20% delta error, not deployable

Pricing Curves: From Single Points to Real-Time Curves



Model	Avg Err(%)	Max Err(%)	Time(s)	Speed-up
MC	-	-	8.53	1.0x
CNN	**1.033**	**1.631**	**0.59**	**14.5x**
LSTM	1.440	3.975	1.56	5.5x
CNN-LSTM	18.373	33.524	4.75	1.8x
TCN	18.657	31.681	6.07	1.4x

Innovation: K/S_0 Feature

- Moneyness ratio as input
- Single model → all strikes & spots
- Batch curve generation

Results (41 points):

- **CNN: 1.033% avg error**
- 100% points < 2% error
- **14.5× speedup** vs MC

Surprising Finding:

- High volatility → *better* accuracy!
- σ=0.35: 1.597% error
- σ=0.15: 1.682% error

Paradigm Shift:
MC batch process → NN real-time curves

Synthetic Data and Prior Findings

Key idea: Since all training data are **synthetically simulated**, the *data generation process itself* becomes part of model design. How we simulate parameters can directly shape behavior.

Prior Findings (Anderson & Ulrych, 2023):

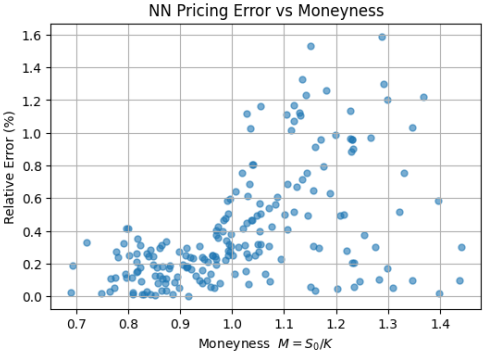
> *"The neural network underperforms in some specific areas of the parameter space. One can tackle this issue by simulating more parameter–price pairs, especially in the under-performing regions."*

Our Question: Does targeted oversampling in such regions improve or harm global model accuracy?

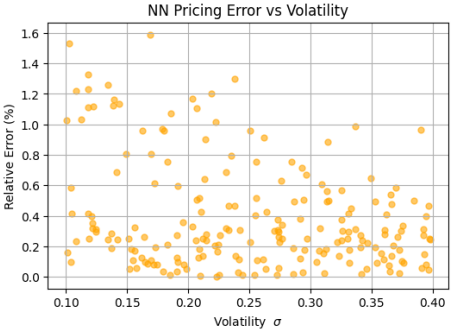
Parameter-Space Error Landscape

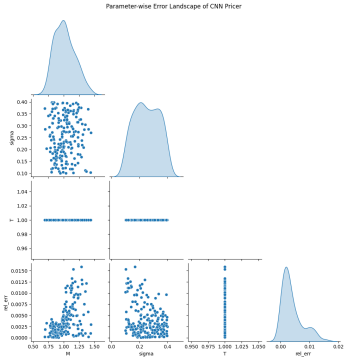
- Evaluate CNN-based neural pricer vs. MC benchmark across 200 random configurations.
- Compute relative error:

$$\text{RelErr} = \frac{|\text{NN} - \text{MC}|}{K}, \quad M = \frac{S_0}{K}$$



Error Sensitivity Analysis





Findings:

- Errors increase in deep ITM ($M > 1.1$) and extreme σ.
- Suggests potential for adaptive sampling — but effect unclear.

Adaptive Sampling: Setup and Results

Procedure:

1. Identify high-error regions:

$$M > 1.1 \quad \text{or} \quad \sigma < 0.15 \text{ or } \sigma > 0.35$$

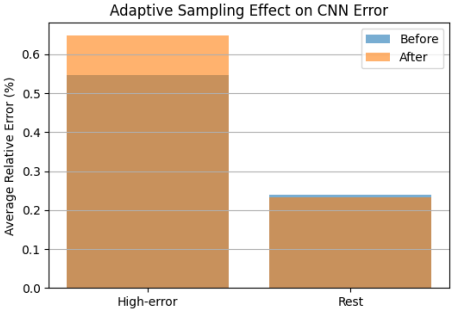
2. Generate additional training samples only within these regions.
3. Retrain the CNN on the combined dataset (45,000 samples total, same parameter bounds).

Findings:

- High-error region: 0.55% → 0.65%
- Rest of space: 0.24% → 0.23%

Interpretation:

In practice, adding more samples in the high-error regions did *not* improve the model's accuracy. This suggests that the CNN may be fundamentally unable to learn the underlying pricing mechanism in these parameter zones.



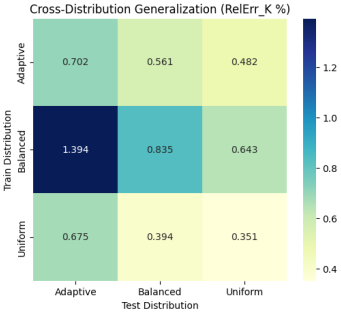
Cross-Distribution Test

Motivation & Setup:

Since all data in this project are *synthetically simulated*, we can freely vary the parameter distribution for both training and testing. This allows us to examine how models trained on one distribution perform when evaluated on another.

- Train three CNN models on:
  - **Uniform:** evenly sampled parameter space;
  - **Adaptive:** focused on high-error regions;
  - **Balanced:** 70% uniform + 30% adaptive.
- Each model is tested on all three distributions.

Purpose:
To test whether focusing on specific regions improves global generalization or introduces bias that reduces overall accuracy.



Result Summary:

- Uniform-trained model performs best on all test sets.
- Adaptive oversampling hurts global accuracy.

Conclusion

Adding more samples in the high-error regions did *not* improve the model's accuracy — it even slightly increased local errors.

The CNN trained under **uniform sampling** performed best across all test sets, demonstrating better global generalization.

Adaptive oversampling may hurts global accuracy and stability.

Thank you!